

Lecture 13 – Space complexity. Circuit complexity.

NTIN071 Automata and Grammars

Jakub Bulín (KTIML MFF UK)

Spring 2026

** Adapted from the Czech-lecture slides by Marta Vomlelová with gratitude.
The translation, some modifications, and all errors are mine.*

Recap of Lecture 12

- time complexity, for TM as well as NTM
- the class P
- the class NP: verifier-based and nondeterminism-based definitions
- polynomial-time reductions
- NP-complete problems
- Cook-Levin theorem: SAT is NP-complete
- the class co-NP
- exponential time complexity

Space complexity

Space complexity

Similarly to time, measure space required for computation:

Definition

The **space complexity** of a [deterministic, single-tape] Turing machine that always halts is $f : \mathbb{N} \rightarrow \mathbb{N}$, where $f(n)$ is the maximum number of cells that M accesses on any input of size n .

For nondeterministic Turing machines we take the maximum over all computation paths.

But this does not work for **sublinear** space complexity, in particular, **logspace** (**L**) and **nondeterministic logspace** (**NL**).

The solution is to have two tapes: a read-only input tape, and a working tape whose space we measure. (If we want output, then a third 'write-only' output tape, only traverse in one direction.)

Classes of space complexity

For $f : \mathbb{N} \rightarrow \mathbb{R}^+$, define the space complexity classes:

$\text{SPACE}(f(n)) = \{L \mid L \text{ decidable by a DTM in space } O(f(n))\}$

$\text{NSPACE}(f(n)) = \{L \mid L \text{ decidable by an NTM in space } O(f(n))\}$

$\text{L} = \text{SPACE}(\log(n))$ note: $\log(n^k) \in O(\log(n))$

$\text{NL} = \text{NSPACE}(\log(n))$

$\text{PSPACE} = \bigcup_k \text{SPACE}(n^k)$

$\text{NPSPACE} = \bigcup_k \text{NSPACE}(n^k)$

Relationship between space and time

- Trivially, $\text{TIME}(f(n)) \subseteq \text{SPACE}(f(n))$ and $\text{NTIME}(f(n)) \subseteq \text{NSPACE}(f(n))$ (at most one cell per step)
- $\text{NTIME}(f(n)) \subseteq \text{SPACE}(f(n))$ (DFS on computation graph, store path from root as sequence of choices, not configs)
- If $f(n) \geq \log(n)$, then $\text{NSPACE}(f(n)) \subseteq \text{TIME}(2^{O(f(n))})$ (only $2^{O(f(n))}$ configs, fully construct computation graph + run BFS)

$$L \subseteq NL \subseteq P \subseteq NP \subseteq PSPACE \subseteq NPSPACE \subseteq EXPTIME$$

Which inclusions are equalities? Which inclusions are strict?

- $PSPACE = NPSPACE$ (Savitch's theorem, coming up)
- $P \neq EXPTIME$ (Time hierarchy theorem, omitted)
- the rest is wide open

Example: NFA accepts everything?

$$\text{ALL}_{\text{NFA}} = \{ \langle A \rangle \mid A \text{ is an NFA such that } L(A) = \Sigma^* \}$$

We can decide $\overline{\text{ALL}_{\text{NFA}}}$ in **nondeterministic linear space**:

- markers on states of A to keep track of set of possible states
- start with marker on start state
- repeat $2^{|Q|}$ times:
 - guess an input symbol $a \in \Sigma$
 - move markers according to transitions (simulate reading a)
 - if no marker on accepting state, accept
- if not accepted so far, reject
- note: $2^{|Q|}$ is enough, more means a loop that can be shortened

Thus $\text{ALL}_{\text{NFA}} \in \text{co-NSPACE}(n) \subseteq \text{co-SPACE}(n^2) = \text{SPACE}(n^2)$

Is ALL_{NFA} in $\text{NP} \cup \text{co-NP}$?

Savitch's theorem

Savitch's theorem

Theorem (Savitch)

For any $f : \mathbb{N} \rightarrow \mathbb{R}^+$ with $f(n) \geq \log(n)$:

$$\text{NSPACE}(f(n)) \subseteq \text{SPACE}(f(n)^2)$$

Corollary

$$\text{PSPACE} = \text{NPSPACE}$$

Idea: Simulate the NTM deterministically by checking reachability in the configuration graph using a recursive algorithm that uses only $O(f(n)^2)$ space.

PATH in $\log^2(n)$ space

Given a directed graph G and $s, t \in V$, is there a path from s to t ?

def PATH(s, t) :

return REACH($s, t, |V|$)

REACH(s, t, k) :

if $k = 0$ **then return** ($s = t$)

if $k = 1$ **then return** ($s = t$) **or** $(s, t) \in E$

for each $u \in V$ **do**

if REACH($s, u, \lfloor k/2 \rfloor$) **and** REACH($u, t, \lceil k/2 \rceil$)

then return true

return false

recursion depth $O(\log(n))$, each level $O(\log(n))$ bits to store k, u

total space $O(\log^2(n))$

Proof of Savitch's theorem

Given NTM M deciding L in space $g(n) \in O(f(n))$, and input w .

Assume M has a single final state and erases tape before accepting, so a single accepting configuration.

Is there a path from C_{init} to C_{accept} in the computation graph?

- $|V| \leq 2^{d \cdot g(n)}$ for some constant d
- $\log |V| \in O(g(n)) \subseteq O(f(n))$
- implement algorithm, start with $\text{REACH}(C_{\text{init}}, C_{\text{accept}}, 2^{d \cdot f(n)})$
- space $O(\log^2 |V|) \subseteq O(f(n)^2)$ □

In the algorithm, we need to construct the value $f(n)$. How?

Either assume the function $f(n)$ is **space-constructible**, i.e., there is a DTM computing $1^n \mapsto f(n)$ and working in space $O(f(n))$.

(Most common functions are space-constructible.)

Or use a trick: Try the value $f(n) = 1$, if the algorithm runs out of space, restart the whole process with $f(n) = 2$, etc.

PSPACE-complete problems

Quantified boolean formulas

A **quantified boolean formula**: $Q_1x_1 Q_2x_2 \cdots Q_kx_k \varphi(x_1, \dots, x_k)$
where $Q_i \in \{\forall, \exists\}$ and φ is a propositional formula.

Definition

TQBF = $\{\langle \varphi \rangle \mid \varphi \text{ is a true QBF sentence}\}$

Note: equivalent to $\text{QSAT} = \{\langle \varphi \rangle \mid \varphi \text{ is satisfiable QBF formula}\}$

Theorem

TQBF is PSPACE-complete (under polynomial-time reductions).

Proof: **TQBF** $\in$ **PSPACE** by space-efficient recursive evaluation

def $\text{EVAL}(\varphi)$:

if no variables **then** evaluate directly

if $\varphi = \exists x \psi$ **then return** $\text{EVAL}(\psi[x \leftarrow 0])$ **or** $\text{EVAL}(\psi[x \leftarrow 1])$

if $\varphi = \forall x \psi$ **then return** $\text{EVAL}(\psi[x \leftarrow 0])$ **and** $\text{EVAL}(\psi[x \leftarrow 1])$

Proof that TQBF is PSPACE-hard

Given $L \in \text{PSPACE}$, decided by DTM M in space $f(n) \in O(n^k)$.

On input w , construct QBF sentence φ true iff M accepts w .

In particular, $\phi_{C_1, C_2, t}$ true iff there is a path of length $\leq t$ from C_1 to C_2 in the computation graph. Then set $\varphi = \phi_{C_{\text{init}}, C_{\text{accept}}, 2^{d \cdot f(n)}}$.

For $k = 0$ easy, for $k = 1$ similar as in the proof of Cook-Levin.

For $k > 1$, **naive approach**: $\exists C_{\text{mid}} \phi_{C_1, C_{\text{mid}}, \lfloor t/2 \rfloor} \wedge \phi_{C_{\text{mid}}, C_2, \lceil t/2 \rceil}$
(where $\exists C_{\text{mid}}$ stands for “there exist vars describing a config” as in Cook-Levin), but this is too long, size $O(t) \subseteq O(2^{d \cdot f(n)})$.

Trick: universal quantifiers to reuse vars at each level of recursion

$$\exists C_{\text{mid}} \forall X, Y \in \{(C_1, C_{\text{mid}}), (C_{\text{mid}}, C_2)\} \phi_{X, Y, \lceil t/2 \rceil}$$

Length remains $O(f(n))$, nesting level $O(\log 2^{d \cdot f(n)}) = O(f(n))$,
total length $O(f^2(n))$. □

$\forall X, Y \in \{(C_1, C_{\text{mid}}), (C_{\text{mid}}, C_2)\}$ is shorthand for $(X = C_1 \wedge Y = C_{\text{mid}}) \vee (X = C_{\text{mid}} \wedge Y = C_2) \rightarrow \dots$

The Game problem

A **game** is $G = \langle V, E, A, B, s \rangle$ where $V = A \dot{\cup} B$ are positions of Players A and B, $E \subseteq V \times V$ are possible moves, and $s \in A$ is the starting position. A player loses if they have no move.

Theorem

$\text{GAME} = \{ \langle G \rangle \mid A \text{ has winning strategy} \}$ is PSPACE-complete.

Proof: GAME is polynomial-time interreducible with TQBF

TQBF $\leq_p$ GAME: given φ , construct G where A chooses values for $\exists$ vars, B for $\forall$ vars; at the end A wins iff φ is true under the chosen assignment

GAME $\leq_p$ TQBF: given G , construct $\varphi = \psi_s$ that is true iff A has winning strategy from s . For each position v , define ψ_v :

$$\psi_v = \bigvee_{(v,u) \in E} \psi_u \text{ if } v \in A, \text{ and } \psi_v = \bigwedge_{(v,u) \in E} \psi_u \text{ if } v \in B.$$

Note: $\bigvee_{\emptyset} = \perp$, $\bigwedge_{\emptyset} = \top$ defines base case for end positions.



Random-access Machines

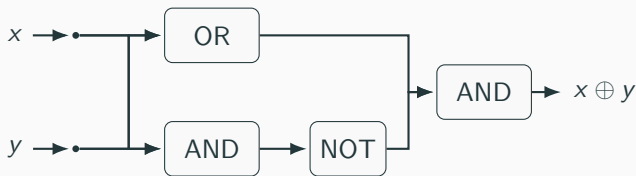
Circuit complexity

Boolean circuits

A **Boolean circuit** is a directed acyclic graph whose nodes (**gates**) are labeled by Boolean functions from some fixed basis, e.g. {AND, OR, NOT}.

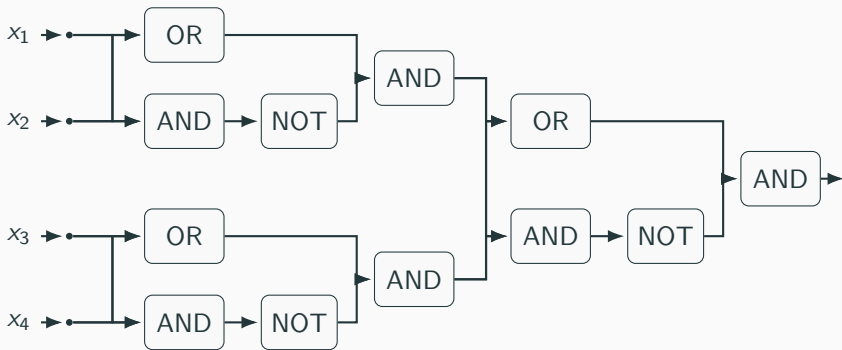
Each circuit C computes a (multivariate) Boolean function $f_C : \{0, 1\}^n \rightarrow \{0, 1\}^m$, where n is the number of **input nodes** (sources), and m is the number of **output gates** (sinks) in C .

Example: parity on 2 bits $x \oplus y = (x \vee y) \wedge \neg(x \wedge y)$



size = 4 (#gates), **depth** = 3 (longest path from input to output)

Example: parity on 4 bits



Optimal size for parity on n bits? $O(n)$

Optimal depth? $O(\log n)$

Counting argument: almost all Boolean functions on n bits require super-polynomial size circuits.

A **circuit family** is a sequence $\{C_n\}_{n \geq 0}$ where C_n has n input nodes.

A circuit family $\{C_n\}$ **decides** a language $L \subseteq \{0, 1\}^*$ if for every n and every $w \in \{0, 1\}^n$,

$$w \in L \iff f_{C_n}(w) = 1$$

Summary of Lecture 13

- space complexity, L, NL, PSPACE, NPSPACE
- relationship between space and time
- Savitch's theorem, $\text{PSPACE} = \text{NPSPACE}$
- PSPACE-complete problems: TQBF, Game
- Random-access Machines
- circuit complexity